OPERATING SYSTEM DESIGN

S Thenmozhi

Department of Computer Applications

OPERATING SYSTEM DESIGN

PROCESS & THREAD MANAGMENT

S Thenmozhi
Department of Computer Applications

OPERATING SYSTEM DESIGN

Background

- Processes can execute concurrently
  - May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes

PES UNIVERSITY
CELEBRATING 50 YEARS

Background

Suppose that we wanted to provide a solution to the producer-consumer problem that fills ***all*** the buffers.

We can do so by having an integer **`counter`** that keeps track of the number of full buffers. Initially, **`counter`** is set to 0.

It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

OPERATING SYSTEM DESIGN

Background

```
while (true) {
        /* produce an item in next_produced */
        while (counter == BUFFER_SIZE) ;
                /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter++;
}
```

```
while (true) {
        while (counter == 0) ;
                /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
         counter--;
        /* consume the item in next_consumed */
}
```

Background

- **counter++**

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **counter--**

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute	`register1 = counter`	{register1 = 5}
S1: producer execute	`register1 = register1 + 1`	{register1 = 6}
S2: consumer execute	`register2 = counter`	{register2 = 5}
S3: consumer execute	`register2 = register2 - 1`	{register2 = 4}
S4: producer execute	`counter = register1`	{counter = **6**}
S5: consumer execute	`counter = register2`	{counter = **4**}

- **Race condition** – a situation when several processes access the same data concurrently, the outcome of the execution depends on the order of the accesses

Critical Section Problem

- Consider system of ***n*** processes {$P_0, P_1, \ldots, P_{n-1}$}
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other is allowed to be in its critical section
- ***Critical section problem*** is to design protocol to solve this difficulty.

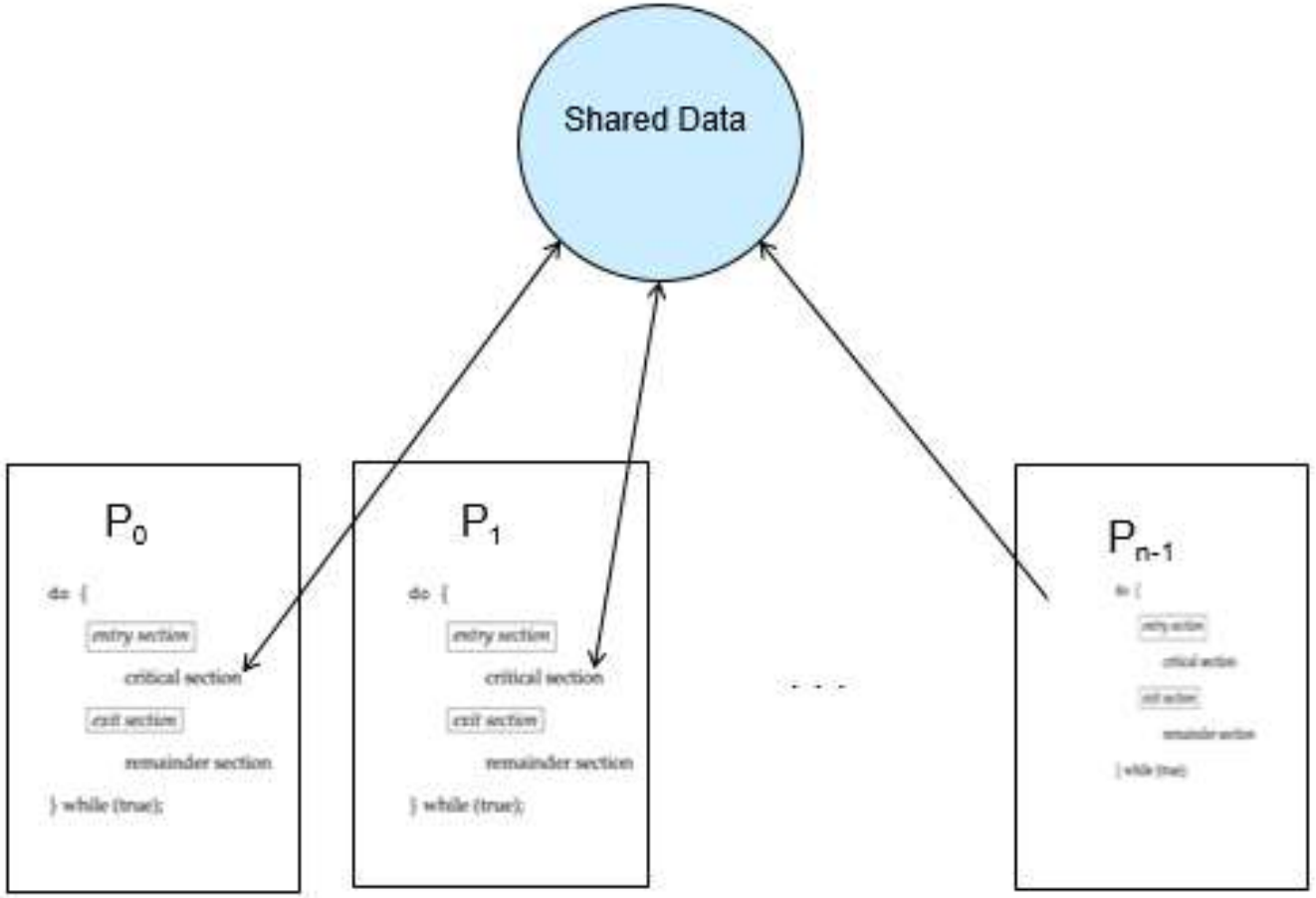
Critical Section Problem

- Each process
  - must ask permission to enter critical section in **entry section**,
  - may follow critical section with **exit section**,
  - then **remainder section** **(**anything else a process does besides using the **critical section)**

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

OPERATING SYSTEM DESIGN

Critical Section Problem



Critical Section Problem

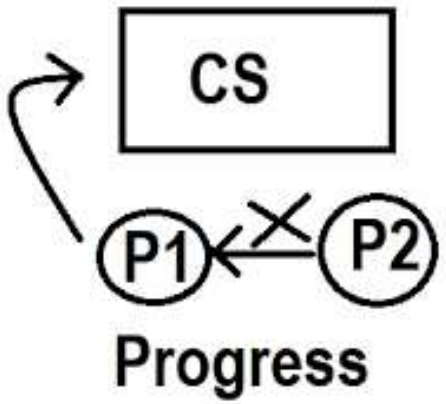
- Each process $\mathbf{P_i}$ has its own critical section
  - Accesses to shared data only in Critical Section
- Lock
  - Shared data is accessible by all
  - Often 1 lock for all n processes
  - Guards access to all critical sections
  - Ensures mutual exclusivity

Critical Section Problem

Solution for a Critical Section Problem must satisfy the following requirements

1. Mutual Exclusion – if process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. Progress – if no process in critical section and if some processes wish to enter into critical sections, then only those processes that are not executing in their remainder section can participate in deciding which will enter its critical section next and this selection cannot be postponed indefinitely

3. Bounded Waiting – There exists a bound or limit on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.



THANK YOU

S Thenmozhi

Department of Computer Applications

thenmozhis@pes.edu

+91 80 6666 3333 Extn 393